Thamill Indian B U
Cohort-15 (AI Engineer)

Day 09 Architecture Strategy & Reflection

AI Engineering Program | Systematic Architecture Decision Framework

PART 1 — ARCHITECTURE DECISION CARDS (SCENARIOS 1 – 4)

Scenario 1 : Small Transformation — PROMPT / CONTEXT

Problem Type: Formatting & Structuring (Behavioral)

Driving Req: Rewrite meeting notes into fixed concise format; zero private data needed.

Rejected Alt: **Fine-Tuning** — Massive over-engineering; requires training dataset and maintenance for basic syntax transformation.

Key Risk: Model drift or instruction-following variance across large prompt context.

Discovery Q: "Are there strict structural schemas (e.g., JSON/Markdown) or rigid output tokens required for downstream automated parsing?"

Scenario 2: Changing Policies — RAG

Problem Type: Knowledge Retrieval + Traceability

Driving Req: 1 00s of internal policies changing regularly; mandatory source citations.

Rejected Alt: **Fine-Tuning** — Catastrophic failure for changing knowledge; fine-tuned weights cannot be updated continuously or cited accurately.

Key Risk: Chunking fragmentation or stale vector indexing leading to incorrect rule citations.

Discovery Q: "What is the update frequency of these policy documents, and are they maintained in a structured repository with document IDs?"

Scenario 5: Eligibility Rule — NO LLM

Problem Type: Deterministic Logic / Compliance

Driving Req: Explicit business rules; 1 00% reproducible, auditable, zero-hallucination mandate.

Rejected Alt: **Prompting / RAG** — LLMs are probabilistic engines and cannot guarantee 1 00% mathematical or logical consistency.

Key Risk: High maintenance cost if business logic changes rapidly in code.

Discovery Q: "Is there any unstructured text evaluation required prior to applying these business rules, or are inputs pure structured data?"

Scenario 6: Private Knowledge + Style — COMBINATION

Problem Type: Knowledge Retrieval + Specialized Behavior

Driving Req: Current internal knowledge with citations + strict specialized formatting that prompting fails at.

Rejected Alt: **RAG Alone** — Standard RAG supplies context but cannot enforce complex specialized formatting behaviors reliably.

Key Risk: Combined operational friction: fine-tuned model drift during API upgrades plus RAG pipeline failures.

Discovery Q: "Can the behavior be enforced via structured JSON schema outputs before investing in fine-tuning a custom model?"

Scenario 7: Tiny Known Context — PROMPT / CONTEXT

Problem Type: Static In-Context Querying

Driving Req: Single short policy; changes rarely; fits completely into immediate request context.

Rejected Alt: **RAG** — Vector DB setup, embedding models, and chunking strategies add zero value and high operational friction for a tiny file.

Key Risk: Prompt inflation if the document grows significantly beyond original length.

Discovery Q: "Will this policy remain isolated, or is it

Scenario 8: Vague Stakeholder Request — DISCOVERY FIRST

Problem Type: Problem Definition & Requirement Scoping

Driving Req: Request to "fine-tune on company data so ChatGPT knows everything."

Rejected Alt: **Premature Architecture Selection** — Selecting Fine-Tuning immediately aligns with a misconception of model training.

Key Risk: Building a fine-tuned model that hallucinates internal facts and cannot update continuously.

PART 2 — DEEP DIVE ON STAGED ARCHITECTURE (SCENARIO 6)

- **Simplest Viable First Version (Stage 1 — Prompt + In-Context RAG):** Implement a standard RAG pipeline (vector search + retrieval) using a frontier model (e.g., GPT-4o) with system instructions and 2-3 few-shot examples enforcing the output format. Add strict JSON Schema validation (Structured Outputs) to guarantee syntactic compliance without fine-tuning.
- **Evidence Justifying Complexity Upgrade (Stage 2 — Fine-Tuned Model + RAG):** Upgrade to a fine-tuned custom model adapter only if systematic evaluation proves:
  1. Standard prompt instructions yield >8% format validation failure rates despite schema enforcement.
  2. High token overhead from system prompts creates prohibitive latency (>2.5s) or financial cost at scale (>50k daily requests).
- **Architecture Transition Triggers:**
  • *Prompting → RAG:* Knowledge base grows beyond what can fit in the context window (~1 28k tokens) or update frequency requires decoupling data from prompts.
  • *RAG → Combination (RAG + Fine-Tuning):* RAG handles factual grounding, but model fails to adopt domain jargon, tone, or highly complex domain structures.
  • *LLM → No LLM:* Output requires zero variance, exact mathematical calculation, or strict legal determinism.

PART 3 — ENGINEERING REASONING

- **Why fine-tuning does not solve frequently changing knowledge:** Fine-tuning modifies parametric weight biases during backpropagation. It is slow, computationally expensive, and susceptible to catastrophic forgetting. It creates a static knowledge snapshot without clear source citations or atomic deletion capabilities when policies change.
- **When supplying context directly is simpler than RAG:** When the total corpus is small (<20k tokens), well-structured, and changes infrequently (e.g., Scenario 7). Direct context injection eliminates vector database maintenance, embedding latency, chunking loss, and retrieval failure modes while guaranteeing 1 00% recall.
- **Why RAG creates unnecessary operational complexity:** RAG adds multiple moving infrastructure pieces: document parsing/OCR, chunking strategies, embedding models, vector database synchronization, hybrid search tuning, reranking steps, and metadata filtering. Every layer introduces a distinct point of failure.
- **Example where No LLM is the stronger architecture:** Payroll Tax Calculation Engine. Calculating employee tax withholdings based on salary bands, state rules, and tax codes must be exact, 1 00% deterministic, fully auditable, and instantly compliant. An LLM introducing probabilistic variation is an unacceptable compliance risk.

PART 4 — FORWARD DEPLOYABLE ENGINEERING (FDE) REFLECTION

- **Discovery Question for Customer Requests:** *"What specific business metric fails if we solve this with a simpler, standard API approach instead of the requested technology?"* (Forces the customer to articulate real functional requirements rather than technology hype).
- **Explaining Simpler Recommendations to Stakeholders:** Reframe simplicity as a business advantage: *"By starting with a direct RAG system rather than fine-tuning, we reduce initial deployment time from 8 weeks to 1 week, cut operational maintenance costs by 60%, and gain the immediate ability to update policies in real-time with verified citations."*
- **Business Evidence Triggering Architecture Upgrades:**
  1. *Systematic Evaluation Failures:* Production test benchmarks fall below acceptable accuracy thresholds (e.g., <95% accuracy).
  2. *Latency / Cost Bottlenecks:* Context window bloat inflates per-query costs or violates SLA response time limits.
  3. *Behavioral Drift:* In-prompt instructions consistently fail to enforce formatting or safety guardrails at high volume.